Data Structures and Algorithms

Lab 02 - Arrays

Instructor: Sidrat-ul-Muntha

READ IT FIRST

Prior to start solving the problems in this assignments, please give full concentration on following points.

1. WORKING – This is individual lab. If you are stuck in a problem contact your teacher, but, in mean time start doing next question (don't waste time).

2. DEADLINE – One Week after assigned this.

3. SUBMISSION – This assignment needs to be submitted in a soft copy.

4. WHERE TO SUBMIT – Please visit your LMS.

5. HOW TO SUBMIT – rename all .java files as task number and make .zip file, name it as 000-00-0000_assignment.zip, and submit .java/.cpp file ONLY.

Task 1: Array Insert, Update and Delete Operations

Create a class named MyArray that represents an array and provides methods to perform basic array operations.

Task Requirements:

- Create a class named **MyArray** with an integer array as a data member.
- Create a **constructor** that takes the size of the array as a parameter.
- Implement an **insert()** method that:
  - Takes an index and a value as parameters.
  - Inserts the value at the specified index.
  - Shifts the existing elements to the right.
- Implement an **update()** method that:
  - Takes an index and a value as parameters.
  - Updates the element at the specified index.
- Implement a **delete()** method that:

    - Takes an index as a parameter.
    - Deletes the element at the specified index.
    - Shifts the remaining elements to the left.
- Implement a **display()** method to print the array elements.
- Create a Main class to test all three operations.

Sample MyArray Class

```
class MyArray {
   private int[] array;
   private int size;

   // Constructor
   public MyArray(int capacity) {
   }

   // Insert an element
   public void insert(int index, int value) {
   }

   // Update an element
   public void update(int index, int value) {
   }

   // Delete an element
   public void delete(int index) {
   }

   // Display array elements
   public void display() {
   }
}
```

Sample Main Class

```
public class Main {
   public static void main(String[] args) {
      MyArray myArray = new MyArray(10);
```

```
        // Test insert
        myArray.insert(0, 10);
        myArray.insert(1, 20);
        myArray.insert(2, 30);

        // Test update
        myArray.update(1, 25);

        // Test delete
        myArray.delete(0);

        // Display array
        myArray.display();
    }
}
```

Task 2: Array Operations Using MyArray Class

Extend the **MyArray** class created in **Task 1** by implementing the following additional methods:

- Implement insertAfter() to insert an element after a specified value.
- Implement updateByValue() to update an element using its existing value.
- Implement deleteByValue() to delete an element using its value.
- Implement search() to search for an element and return its index.
- The search() method should return -1 if the element is not found.
- Test all the new methods in the Main class.

Sample MyArrayClass

```
class MyArray {

    // Insert an element after a specific value
    public void insertAfter(int afterValue, int value) {

    }

    // Update an element using its value
    public void updateByValue(int oldValue, int newValue) {
```

```
    }

    // Delete an element using its value
    public void deleteByValue(int value) {

    }

    // Search for an element
    public int search(int value) {

        return -1;
    }
}
```

Sample Main Class

```
public class Main {
    public static void main(String[] args) {
        MyArray myArray = new MyArray(10);

            // Test insert
              myArray.insert(0, 10);
              myArray.insert(1, 20);
              myArray.insert(2, 30);

            // Insert after a value
              myArray.insertAfter(20, 25);

             // Update using value
              myArray.updateByValue(30, 35);

              // Delete using value
              myArray.deleteByValue(25);

              // Search for a value
              myArray.search(20);

    }
}
```

Task 3: Two Sum

Given an array of integers nums and an integer target, find the **indices of the two elements** whose values add up to the given target.

Requirements:

- Create a method that takes an integer array nums and an integer target as parameters.
- Find two different elements in the array whose sum is equal to target.
- Return the indices of those two elements.
- You may assume that each input has **exactly one solution**.
- The same array element cannot be used twice.
- The indices can be returned in any order.
- Test the method using the given examples.

Example

Input:
nums = [2, 7, 11, 15]
target = 9
Output:
[0, 1]

Sample Method

```
public static int[] twoSum(int[] nums, int target) {

   // Write your logic here

   return new int[]{};
}
```

Task 4: Rotate Array

Given an integer array nums, rotate the array to the **right by k steps**, where k is a non-negative integer.

Requirements:

- Create a method that takes an integer array nums and an integer k as parameters.
- Rotate the array to the right by k positions.

- The rotation should be performed on the same array.
- If k is greater than the length of the array, handle it appropriately.
- Test the method using the given examples.

Example

Input:
nums = [1, 2, 3, 4, 5, 6, 7]
k = 3
Output:
[5, 6, 7, 1, 2, 3, 4]

Sample Method

```
public static void rotate(int[] nums, int k) {

    // Write your logic here

}
```
